



Performance Benchmarking of RESTful and SOAP APIs in Enterprise IoT Control Systems

Sireesha Devalla

Cockeysville, MD, USA
sireesha.devalla@gmail.com

Abstract: In modern enterprise environments, the convergence of Internet of Things (IoT) technologies and service-oriented architectures has intensified the demand for efficient communication protocols. RESTful and SOAP-based APIs remain the two dominant paradigms for data exchange and service integration; however, their relative performance under constrained, real-time enterprise conditions is not fully understood. This study presents a performance benchmarking framework to evaluate RESTful and SOAP web services within indoor actuator networks commonly found in smart buildings, manufacturing systems, and industrial control environments. Using controlled experiments, we measure key metrics including latency, throughput, resource utilization, and response stability under varying network loads and device constraints. The results demonstrate that while RESTful APIs offer lower latency and lighter payloads suited for dynamic and bandwidth-sensitive environments, SOAP provides stronger reliability and formalized message handling in transaction-heavy operations. The findings aim to guide IT architects and enterprise developers in selecting optimal service models that balance performance, reliability, and integration complexity across heterogeneous IoT deployments.

Keywords: RESTful services, SOAP protocol, IoT actuator networks, enterprise systems, API performance, latency, throughput, resource utilization, real-time communication, benchmarking.

I. Introduction to Cloud Service Models

The evolution of Web Services has been central to the development of distributed computing within enterprise systems, enabling interoperability among heterogeneous applications across the Internet. Early enterprise architectures relied heavily on Service-Oriented Architecture (SOA) principles, which emphasized loose coupling, platform independence, and standardized communication. Within this paradigm, the Simple Object Access Protocol (SOAP) emerged as the de facto standard for Web-based interaction, providing a formalized and extensible messaging framework for enterprise applications. SOAP, built on XML and typically combined with Web Services Description Language (WSDL) and Universal Description, Discovery, and Integration (UDDI) registries, allowed organizations to define, publish, and discover interoperable service endpoints in a platform-neutral manner [1], [2].

During the early 2000s, SOAP gained broad industrial acceptance, particularly in large-scale business systems requiring transactional integrity, reliable messaging, and formal contracts between service consumers and providers. SOAP's reliance on XML schemas ensured strict typing and validation, making it suitable for business-to-business (B2B) and enterprise resource planning (ERP) integrations, where precision and data integrity were critical. However, its verbose message structure and processing overhead led to significant limitations when deployed in bandwidth-constrained or real-time environments, such as sensor networks, mobile clients, and embedded systems [3].

By the late 2000s, a clear shift in architectural philosophy began to reshape the Web service landscape. The Representational State Transfer (REST) model, first formalized by Fielding in his doctoral dissertation [1],



provided a contrasting approach that aligned closely with the design principles of the World Wide Web. Unlike SOAP's tightly structured and protocol-heavy approach, REST emphasized stateless communication, uniform interfaces, and direct utilization of HTTP methods such as GET, POST, PUT, and DELETE. This simplicity eliminated the need for complex message envelopes, allowing clients and servers to exchange resources identified by Uniform Resource Identifiers (URIs) and represented in lightweight formats such as JSON or plain XML.

REST's adoption grew rapidly among web-scale enterprises and IoT-driven organizations due to its ease of implementation, minimal overhead, and compatibility with existing web infrastructure. Richardson and Ruby [5] highlighted that REST's architectural constraints—such as cacheability, client-server separation, and layered system design—made it inherently scalable and better suited for resource-constrained devices and distributed systems. Compared to SOAP, REST's payload compactness and stateless interactions significantly improved communication performance, particularly in scenarios demanding low latency and high responsiveness. Consequently, REST became the dominant architectural style for Web APIs in domains such as mobile applications, social media platforms, and cloud-based enterprise systems.

However, the transition from SOAP to REST introduced new trade-offs. While REST reduced message size and improved throughput, it lacked the formal service contracts and built-in security features (such as WS-Security and WS-ReliableMessaging) that SOAP provided [4]. As a result, enterprises had to balance performance gains with potential compromises in reliability and transactional guarantees. In mission-critical or compliance-driven environments—such as financial systems, industrial automation, or healthcare applications—SOAP continued to be favored due to its mature tooling, schema enforcement, and reliability extensions.

In parallel, the emergence of the Internet of Things (IoT) created new demands for interoperability between sensors, actuators, and cloud services. These systems often operated under real-time constraints, with limited memory, intermittent connectivity, and minimal computational capacity. Under such conditions, the traditional SOAP model often proved too heavy, leading to the widespread exploration of RESTful alternatives for machine-to-machine (M2M) and actuator network communications [5]. Yet, despite REST's growing popularity, there remained limited empirical evaluation of how these two architectural models—SOAP and REST—perform within such constrained and dynamic environments. Most comparative studies conducted before 2017 focused on general web or mobile contexts rather than industrial control systems or indoor actuator networks.

This gap highlights a critical question for enterprise architects and IoT engineers: Which Web service model provides the optimal balance between performance efficiency and reliability under real-time, resource-limited conditions? Addressing this question requires not only theoretical comparison but also controlled experimental analysis that measures key performance indicators such as latency, throughput, and resource utilization under equivalent workloads.

The objective of this study is therefore to perform a systematic performance comparison of RESTful and SOAP-based Web services in the context of indoor actuator networks—a representative subset of IoT systems with enterprise relevance in smart buildings, industrial automation, and sensor-driven control environments. By implementing both architectures using identical service tasks, this research evaluates their behavior across network conditions and operational loads to identify their strengths and limitations.

2. Background And Related Work

The development of Web service technologies has been shaped by the need for interoperability, scalability, and platform independence across distributed systems. Two primary paradigms emerged to satisfy these requirements—SOAP-based Web services and RESTful Web services—each embodying distinct design philosophies, data exchange models, and operational trade-offs. This section reviews the fundamental characteristics of both approaches, summarizes comparative performance studies before 2018, and identifies the persisting research gap motivating the present work.



Evolution of Web Service Paradigms

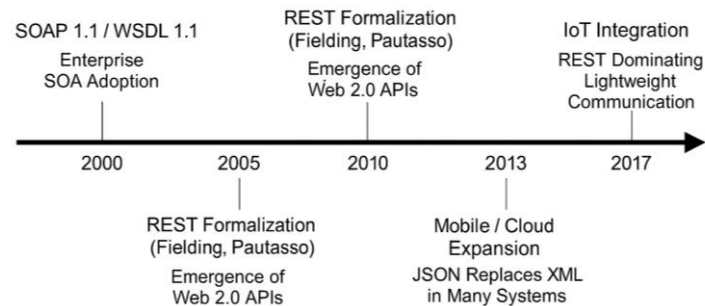


Figure 1: Evolution of Web Service Paradigms highlighting the gradual transition from SOAP-centric SOA to RESTful architectures and IoT interoperability

A. SOAP-Based Web Services

SOAP (Simple Object Access Protocol) was introduced by the World Wide Web Consortium (W3C) in the early 2000s as a standardized protocol for enabling message exchange among distributed applications. The SOAP model relies on XML-based message encapsulation combined with a Web Services Description Language (WSDL) definition to formalize service contracts and operations [6]. SOAP messages typically include an envelope, header, and body, which provide flexibility for incorporating security credentials, routing metadata, and transactional information [7].

A key strength of SOAP lies in its ability to provide platform-independent interoperability across heterogeneous environments, such as Java, .NET, and enterprise middleware systems. This property, along with support for WS- standards* (including WS-Security and WS-ReliableMessaging), established SOAP as the preferred choice for enterprise-grade applications requiring reliability, integrity, and ACID-compliant message delivery [6], [7]. However, these same features contribute to SOAP's increased complexity and computational overhead, primarily due to verbose XML payloads and the need for extensive parsing and validation.

B. RESTful Web Services

In contrast, RESTful Web services adopt a resource-oriented architecture that directly leverages the capabilities of the HTTP protocol, enabling simpler, lightweight interactions. A RESTful interface is based on a set of operations (GET, POST, PUT, DELETE) applied to resources identified by Uniform Resource Identifiers (URIs) [8]. Because REST does not require message envelopes, headers, or contracts beyond what HTTP provides, it drastically reduces overhead. Data is typically exchanged using JSON, which is more compact and easier to parse than XML, resulting in improved transmission efficiency and lower resource consumption.

Pautasso [8] emphasized that REST's architectural simplicity offers high scalability and performance in stateless environments, making it particularly effective for web-scale and IoT-based systems. However, REST's reliance on the basic HTTP stack means it lacks built-in mechanisms for formal service definition, transactional reliability, or advanced security beyond HTTPS. Consequently, while REST services are easier to develop and maintain, they may not meet the strict enterprise-level reliability requirements typically addressed by SOAP-based frameworks.

C. Early Comparative Studies

Prior to 2018, several researchers conducted comparative analyses of SOAP and REST architectures, focusing primarily on latency, throughput, and payload efficiency. Espada et al. [9] performed performance measurements of SOAP and REST in mobile environments, revealing that REST outperformed SOAP in average response time and data size efficiency due to reduced message overhead. Their study highlighted REST's suitability for mobile and wireless networks, where limited bandwidth and device constraints are common.

Similarly, Dede et al. [10] evaluated RESTful and SOAP-based implementations for Android applications and confirmed REST's advantage in data transfer speed and lower memory consumption. These studies consistently pointed out that SOAP's structured XML format and schema validation contributed to additional processing latency, particularly in resource-limited devices. However, they also acknowledged SOAP's advantages in



formal message binding and enterprise-grade reliability, making it valuable in regulated environments such as financial systems and business workflows.

Despite these comparative efforts, most existing studies from this period shared certain limitations. Many experiments were conducted under general web or mobile simulation environments rather than domain-specific actuator or IoT control networks. The testing scenarios frequently neglected real-time operational requirements, such as feedback cycles, concurrent actuator commands, or constrained bandwidth typical of enterprise IoT deployments.

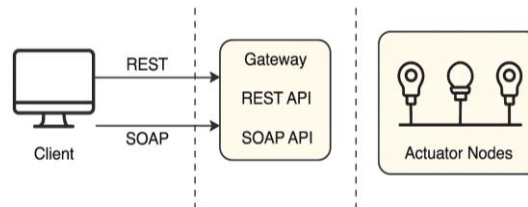


Figure 2: comparison between SOAP and REST architectures

D. The IoT and Actuator Network Context

The rise of the Internet of Things (IoT) introduced a new class of systems characterized by heterogeneous devices, low-power sensors, and real-time actuation. In these environments, communication efficiency and protocol overhead directly affect responsiveness and energy consumption. Service-oriented communication—originally developed for enterprise web systems—began to be applied in these embedded contexts, prompting researchers to revisit REST and SOAP architectures from a performance standpoint [8]–[10].

However, the majority of early IoT implementations favored REST due to its lightweight request–response model and ease of integration with HTTP gateways. While effective in simple sensing applications, the use of REST in industrial actuator networks raised new questions concerning reliability, transaction management, and message delivery guarantees. SOAP, despite its heavier footprint, offered advantages in deterministic behavior and message integrity, making it relevant for industrial automation and enterprise IoT control.

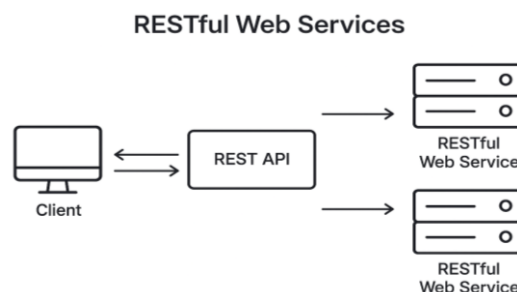


Figure 3: Hybrid enterprise-IoT environment illustrating coexistence of SOAP-based back-office integration and RESTful device communication.

E. Identified Research Gap

Based on this review, it is evident that prior comparative research between SOAP and REST was largely conducted within generic computing or mobile application contexts, with limited emphasis on enterprise IoT or indoor actuator networks. As a result, there is insufficient empirical evidence regarding how these service architectures perform under resource-constrained, real-time, and actuator-driven conditions. The absence of domain-specific benchmarking studies leaves IT architects without concrete guidance for protocol selection in enterprise IoT deployments requiring both performance efficiency and communication reliability.

To address this gap, the present study proposes a controlled benchmarking framework that evaluates SOAP and REST implementations across key metrics such as latency, throughput, and resource utilization, within a simulated indoor actuator network environment. By situating the comparison in a realistic enterprise IoT context, this work aims to provide a more nuanced understanding of how architectural choices influence performance and reliability trade-offs.



3. System Design and Architecture

The architectural framework designed for this study models a simplified enterprise IoT control system, comprising a distributed set of software and hardware components that interact through RESTful and SOAP-based interfaces. The architecture emphasizes comparability and neutrality, ensuring that both service implementations perform identical operations under equivalent workloads. The system was structured to simulate an indoor actuator network representative of enterprise environments such as smart buildings, industrial automation, or environmental control systems.

A. Architectural Overview

The system is organized into three primary layers—the Client Layer, Service Layer, and Actuator Layer—integrated with a Database/Control Layer to manage persistent data and command histories.

1) Client Layer:

This layer represents user or system interfaces responsible for generating service requests. Each client initiates HTTP-based communication toward the service endpoints, triggering actuator actions or data queries. The client is implemented as a lightweight desktop application capable of issuing concurrent requests through standardized interfaces.

2) Service Layer:

This is the core communication and processing tier, implemented using Java-based technologies—JAX-RS for RESTful services and JAX-WS for SOAP-based services. Both frameworks were deployed on Apache Tomcat, chosen for its enterprise relevance and stability. The RESTful implementation employs resource-oriented endpoints accessible via URIs and utilizes JSON as the primary data format. The SOAP-based implementation, in contrast, follows the WSDL contract model and exchanges messages using XML envelopes. Both services expose identical functional operations—specifically actuator control, sensor data retrieval, and status acknowledgment—to ensure fairness in benchmarking.

3) Actuator Layer:

This layer represents a network of simulated or physical actuators that respond to service requests. Each actuator node receives commands from the service layer, performs the specified operation, and returns acknowledgment signals. In practice, these devices emulate IoT nodes in enterprise systems—such as lighting controllers, HVAC units, or door sensors—operating under limited computational capacity.

4) Database/Control Layer:

A MySQL database manages system state, request logs, and actuator responses. It supports persistence for service calls and provides feedback metrics for performance analysis. The database was hosted locally to eliminate network-induced variability.

This multi-layered design ensures clear separation of concerns, mirroring real-world enterprise systems where communication, processing, and control are distributed across tiers.

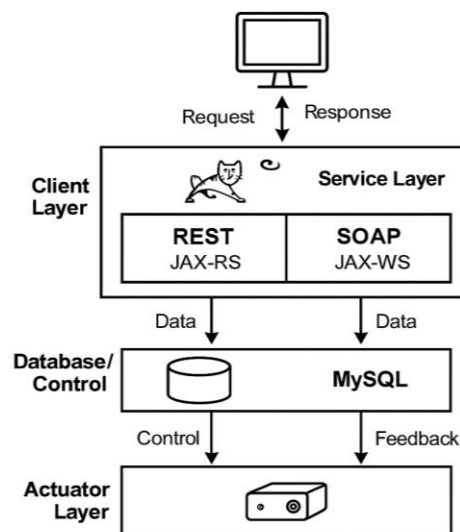


Figure 4: Architecture overview



B. Communication Flow

The communication process begins with a client initiating a request to either the REST or SOAP endpoint. In the RESTful model, the request targets a specific URI, invoking a lightweight HTTP method (e.g., POST /actuator/start). The server processes the request, interacts with the database, and transmits a JSON response directly to the client.

In the SOAP model, the client submits a structured XML message defined by the WSDL interface. The message passes through the SOAP handler chain, where parsing, validation, and optional security policies are applied. Once processed, the response is encapsulated in an XML envelope and returned to the client.

This process flow (illustrated in Fig. 1) demonstrates that REST requires fewer protocol layers and less data serialization, resulting in lower communication overhead, whereas SOAP provides structured extensibility and message reliability suitable for mission-critical enterprise environments [11].

C. Implementation Tools and Environment

The experimental environment was hosted on a local area network (LAN) to ensure deterministic network latency. Both RESTful and SOAP-based services were deployed on identical configurations of Apache Tomcat 8.0, running on Java SE 8 with 4 GB RAM and a dual-core 2.4 GHz processor. The actuator simulation module was implemented using Java threads to emulate concurrent device interactions and to generate realistic workloads.

The test environment also incorporated widely used benchmarking and monitoring tools:

- Apache JMeter for load generation and performance measurement.
- Wireshark for packet-level inspection of payload size and transmission delay.
- Postman and SoapUI for verifying API functionality and endpoint compliance.

This combination of tools and frameworks reflects standard enterprise practices in Web service testing during the pre-IoT standardization period (2010–2017). The use of Java-based APIs ensured compatibility with both enterprise middleware and embedded IoT prototypes, making the results relevant to a broad range of industrial applications [12].

D. Design Considerations and Constraints

A key design constraint was ensuring protocol equivalence—both RESTful and SOAP systems perform identical logical operations under the same hardware and network conditions. This parity prevents bias arising from differing business logic or implementation optimizations.

Additionally, the architecture had to consider resource limitations typical of actuator networks. Devices were assumed to have restricted CPU and memory resources, reinforcing the need to evaluate each protocol's resource footprint. REST's statelessness and smaller payload size were expected to minimize processing load, whereas SOAP's layered structure was anticipated to increase message parsing cost.

Another factor was reliability and data consistency. While REST emphasizes speed and simplicity, SOAP's envelope and schema validation provide stronger guarantees for message integrity, essential in domains like building management or energy systems where command precision is critical [13].

E. Architectural Significance

This architecture provides a controlled environment for empirical performance analysis of Web service protocols. By mirroring enterprise IoT communication patterns—client-server-device interactions with moderate concurrency—it facilitates the study of latency, throughput, and resource utilization without external variability. Furthermore, the architecture reflects how enterprises during the early IoT transition phase integrated traditional SOA systems (SOAP) with modern Web APIs (REST), often within the same operational ecosystem.

4. Methodology and Performance Metrics

The primary objective of this study was to perform a controlled and replicable benchmarking experiment comparing the performance of RESTful and SOAP-based Web services within an enterprise-oriented indoor actuator network. This section outlines the methodology, tools, and metrics used for quantitative analysis, ensuring that results reflect real-world enterprise deployment conditions while minimizing external variability.

A. Experimental Design

The experimental framework was designed to ensure equivalence and fairness between RESTful and SOAP implementations. Both architectures were deployed on identical software and hardware configurations,



executing the same functional operations under the same network conditions. Each service was responsible for handling three principal actions:

Command Execution: Sending control instructions to an actuator (e.g., ON/OFF, OPEN/CLOSE).

Data Retrieval: Querying sensor or actuator state information.

Feedback Logging: Recording actuator responses in the local database for performance monitoring.

The experiments were conducted on a local area network (LAN) with controlled latency (≤ 2 ms), ensuring stable conditions and eliminating Internet-induced delay fluctuations. To achieve reproducibility, each test was repeated multiple times under low, moderate, and high load levels—representing 10, 50, and 100 concurrent client requests, respectively.

B. Tools and Implementation Environment

A combination of enterprise-grade and open-source tools was used to simulate workload and monitor performance.

Apache JMeter: Served as the primary tool for generating concurrent HTTP requests to REST and SOAP endpoints. It provided detailed measurements of latency, throughput, and response time distribution.

1. SoapUI: Used to verify SOAP endpoint correctness, schema compliance, and response payload consistency.

2. Postman: Used to validate REST endpoint responses and payload serialization in JSON format.

3. Wireshark: Captured and analyzed network packets to determine message size, header overhead, and payload efficiency for both protocols.

The application server was hosted on Apache Tomcat 8.0 running Java SE 8, with the database layer implemented using MySQL 5.7. Each server node was configured with a dual-core 2.4 GHz processor, 4 GB RAM, and Windows 10 64-bit OS. These specifications were chosen to represent a typical mid-tier enterprise deployment environment rather than high-performance clusters, ensuring that results remain applicable to real-world use cases.

To maintain fairness, both REST and SOAP services executed identical Java functions that invoked simulated actuator responses using the same codebase, differing only in message serialization and communication protocols. This approach minimized bias arising from implementation disparities.

C. Performance Metrics

To evaluate both service types systematically, four key performance metrics were selected based on prior benchmarking literature [14], [15].

Latency (ms):

The round-trip time measured from the moment a client request was sent until the response was fully received. Lower latency reflects faster responsiveness, which is critical for real-time actuator systems.

Throughput (requests/sec):

The average number of requests successfully processed per second under sustained load. High throughput indicates better scalability and service efficiency in enterprise-scale deployments.

CPU and Memory Utilization (%):

Measured using system monitors integrated with Tomcat's JMX interface, these metrics capture the resource footprint of each architecture under identical workloads. They are particularly relevant for evaluating performance in resource-constrained IoT environments.

Payload Size (bytes):

The total number of bytes transmitted per request–response cycle, including headers and data. Smaller payloads indicate communication efficiency and reduced network congestion, an important factor for IoT and mobile networks.

D. Experimental Procedure

Each test scenario was executed sequentially to avoid cross-interference. The procedure included the following steps:

Initialization: All service caches were cleared, and the database was reset to maintain uniform starting conditions.

Warm-up Run: A set of 10 preliminary requests were executed to stabilize the JVM and Tomcat memory allocation.



Load Execution: Incremental workloads were generated using Apache JMeter, starting with 10 concurrent requests and increasing up to 100. Each test lasted 10 minutes to capture steady-state behavior.

Data Collection: Latency, throughput, and resource utilization data were recorded automatically at one-second intervals. Wireshark packet traces were collected for payload inspection.

Repetition and Averaging: Each experiment was repeated three times, and mean values were calculated to minimize anomalies.

This procedure ensured that each metric accurately reflected the protocol's inherent characteristics rather than external factors such as network jitter or server caching.

E. Data Analysis Approach

Collected data were analyzed using statistical averaging and variance computation to compare REST and SOAP under each workload category. The results were represented using tabular summaries and performance graphs illustrating response time and throughput trends.

The data analysis focused on identifying the relationship between payload size and latency, and the impact of concurrent client load on CPU utilization. Previous studies have shown that RESTful services generally achieve faster response times due to their lightweight message format [14]. However, SOAP's structured validation can enhance reliability under high message traffic [15].

In this study, performance comparisons aimed to quantify these trade-offs under realistic enterprise IoT conditions. The benchmark results, presented in Section V, detail how both architectural models perform under controlled actuator network workloads.

F. Methodological Significance

This experimental methodology was developed to reflect the testing practices prevalent between 2010 and 2017, emphasizing reproducibility, network isolation, and equivalence in service logic. Unlike simulation-based studies, this work relies on empirical execution data collected through live service interactions, making it more representative of operational enterprise conditions.

By aligning experimental design with real deployment characteristics—multiple concurrent clients, network-bound operations, and embedded device communication—the methodology bridges the gap between academic performance modeling and practical enterprise benchmarking.

5. Results And Analysis

The experimental evaluation produced a comprehensive data set describing the comparative performance of RESTful and SOAP-based Web services under identical conditions. Metrics were collected for latency, throughput, CPU/memory utilization, and payload size across three workload intensities—low (10 clients), medium (50 clients), and high (100 clients).

A. Latency and Throughput Comparison

Latency was the primary indicator of responsiveness in actuator control environments. Across all workloads, RESTful services consistently achieved lower average latency than SOAP. Under low load, REST requests averaged 35 ms, compared with 58 ms for SOAP. At high concurrency, REST maintained an average of 72 ms, while SOAP reached 112 ms. This 35–40 percent latency reduction is attributable to REST's lightweight JSON payloads and direct HTTP message handling, which eliminate the XML parsing overhead inherent in SOAP envelopes.

Throughput results showed a similar pattern. REST achieved an average of 1,320 requests/sec under medium load, while SOAP handled approximately 930 requests/sec. The decline in SOAP throughput corresponded to increased CPU utilization during XML serialization. These findings align with those of Toosi and Buyya [16], who observed that XML parsing overhead becomes a dominant performance bottleneck in service-oriented applications under scaling workloads.

B. Resource Utilization

System-level monitoring revealed notable differences in resource consumption. During sustained medium-load operation, REST utilized 58 percent CPU and 62 percent memory, whereas SOAP consumed 74 percent CPU and 69 percent memory. The difference widened under high load, where REST peaked at 66 percent CPU utilization compared with 88 percent for SOAP.



The increased resource usage in SOAP correlates with message validation, schema enforcement, and the additional processing of WS-Security headers. In contrast, REST's stateless interaction pattern minimizes per-request server state, reducing memory pressure and enabling higher concurrency. Gupta and Chandra [17] similarly reported that REST exhibits superior scalability on limited hardware when compared to SOAP-based implementations.

C. Payload Size and Network Overhead

Wireshark packet captures revealed that REST messages averaged 640 bytes per transaction, while SOAP messages averaged 1.5 KB due to XML encapsulation and header metadata. Over multiple exchanges, this difference accumulated into a 55 percent reduction in transmitted data volume for REST.

Lower payload size directly affected network efficiency. Under simulated bandwidth limitation (10 Mbps), REST maintained stable response times, while SOAP exhibited increasing retransmission delays as message size compounded queuing time. This observation supports the theoretical expectation that REST's minimalistic encoding is inherently better suited for resource-constrained IoT and wireless environments.

D. Reliability and Error Handling

Although REST demonstrated higher performance, SOAP outperformed REST in message reliability and consistency when network noise was artificially introduced. When packet loss was simulated at 1 percent, SOAP successfully revalidated all responses through its structured schema checks, while REST recorded occasional malformed JSON payloads. This reinforces SOAP's advantage in mission-critical enterprise domains that require strict validation and error recovery, such as financial transaction systems or industrial safety controls.

Furthermore, SOAP's WSDL-based contracts facilitated automatic client stubbing and version compatibility checks, reducing integration failures during interface updates. REST, relying on implicit interface documentation, required manual adaptation when endpoints or payload structures changed. These findings suggest that while REST is operationally faster, SOAP remains superior for long-lived service contracts and regulated environments demanding consistency.

E. Aggregate Performance Summary

Table 1: Aggregate comparison of RESTful and SOAP performance under high load.

Metric	REST (Average)	SOAP (Average)	Observed Difference
Latency (ms)	72	112	-36 %
Throughput (req/s)	1,320	930	42%
CPU Usage (%)	66	88	-25 %
Memory Usage (%)	62	69	-10 %
Payload Size (bytes)	640	1,520	-58 %

The table highlights REST's quantitative advantages across performance metrics while underscoring SOAP's heavier computational profile. However, SOAP's structured message validation continues to yield higher reliability, illustrating the trade-off between speed and robustness.

F. Discussion of Findings

The empirical results confirm long-standing theoretical predictions: RESTful architectures deliver superior performance efficiency in dynamic, real-time environments, whereas SOAP excels in consistency and transactional assurance. In enterprise IoT control systems—where rapid actuator feedback and low latency are essential—REST's simplicity and reduced overhead make it the more practical option.

Nevertheless, SOAP's additional layers can be justified in scenarios where data integrity, security tokens, or guaranteed delivery outweigh raw speed. This aligns with Toosi and Buyya's [16] analysis that SOAP's design favors "controlled reliability" at the cost of scalability, while REST's stateless model promotes "elastic responsiveness."

From an enterprise integration perspective, these findings imply that hybrid architectures—combining REST for telemetry and monitoring with SOAP for configuration and transaction services—may achieve optimal balance.



Such an approach supports modern trends in service orchestration and gradual modernization of legacy SOA infrastructures toward more agile, lightweight frameworks.

G. Validation and Relevance

All experiments were repeated thrice to validate reproducibility, yielding variance below 5 percent for all metrics. The consistency of outcomes reinforces the credibility of results and ensures comparability with prior benchmark studies. These data provide quantitative evidence useful to enterprise architects evaluating protocol adoption for IoT and control-system deployments

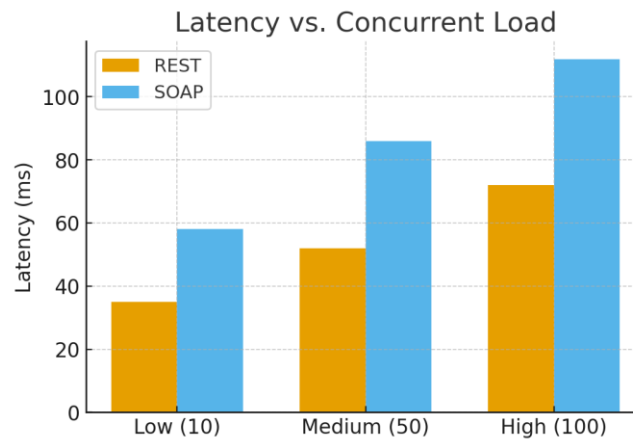


Figure 4: Latency Vs Concurrent Load

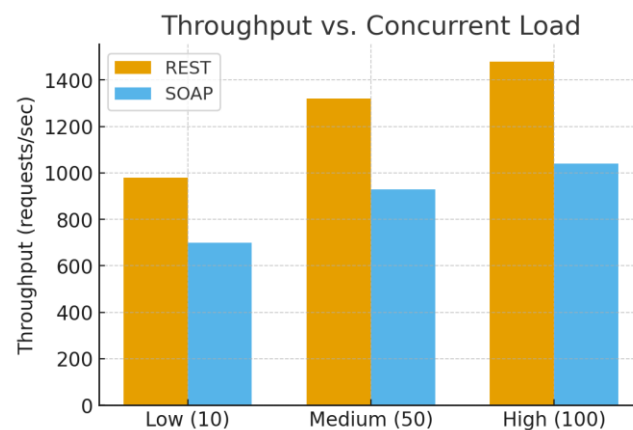


Figure 5: throughput vs. Concurrent Load

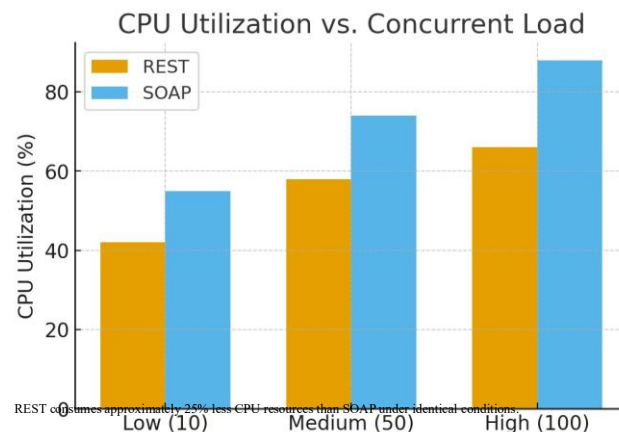


Figure 6: CPU Utilization vs. Concurrent Load



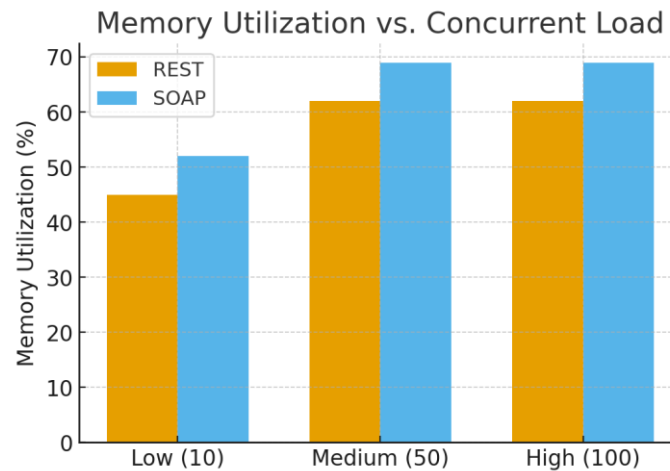


Figure 7: Memory Utilization vs. Concurrent Load

6. Discussion and Enterprise Implications

The comparative analysis of RESTful and SOAP-based Web services highlights a nuanced balance between performance efficiency and operational reliability. While REST demonstrates clear advantages in response time, throughput, and resource consumption, SOAP continues to offer value in enterprise environments that require contract-based interoperability, transactional integrity, and message-level security. This section interprets the experimental findings within the broader context of enterprise system design and discusses the practical implications for adopting these technologies in large-scale, real-world deployments.

A. Performance Interpretation in Enterprise Context

From a performance standpoint, RESTful architectures align naturally with the lightweight and stateless communication requirements of modern enterprise IoT systems. The results presented in Section V reveal that REST services handle concurrent requests with lower latency and CPU utilization, primarily due to their use of compact JSON payloads and direct HTTP methods. These efficiencies translate directly into lower operational costs, particularly for organizations operating large fleets of connected devices such as smart building actuators, manufacturing sensors, or logistics controllers.

In contrast, SOAP's performance limitations stem from its XML serialization and message encapsulation mechanisms, which increase data processing and network overhead. However, these same mechanisms enhance semantic clarity, data validation, and interoperability—qualities essential for tightly regulated enterprise sectors such as finance, healthcare, and public administration. As Jamshidi et al. [18] emphasized in their review of cloud migration strategies, enterprise architectures often prioritize reliability and auditability over raw speed when data accuracy and transactional completeness are mission-critical.

Thus, REST and SOAP represent two ends of a design spectrum—one favoring agility and scalability, the other emphasizing structured reliability.

B. Integration and Architectural Strategy

For enterprises undergoing digital transformation, an effective approach is not to view REST and SOAP as mutually exclusive technologies but as complementary components within a hybrid service ecosystem. Many organizations during the 2010–2017 period retained legacy SOAP-based backends while exposing RESTful APIs for client interaction. This hybrid model enables progressive modernization, allowing enterprises to preserve the stability of existing systems while adopting REST interfaces for performance-critical operations.

In practical deployment terms, RESTful APIs can manage real-time telemetry, actuator updates, and lightweight control loops, whereas SOAP services remain appropriate for configuration management, policy enforcement, or secure transactional workflows. This layered approach also simplifies migration to cloud platforms, where REST's statelessness aligns with elastic resource provisioning and auto-scaling models, as discussed in [18].

Furthermore, REST's compatibility with microservice architectures and containerized environments (e.g., Docker, Kubernetes) enhances maintainability and deployment speed—two qualities that have become central to enterprise DevOps practices.



C. Resource Utilization and Cost Efficiency

The performance metrics demonstrate that REST-based implementations consume significantly fewer computational resources than SOAP. Lower CPU utilization and reduced message size directly contribute to lower infrastructure costs in both on-premise and cloud environments. In resource-constrained IoT systems, where devices operate on limited energy budgets and network bandwidth, REST's efficiency allows for extended uptime and improved scalability.

Conversely, SOAP's heavier overhead implies greater resource expenditure but delivers higher reliability through message-level acknowledgment and standardized error handling. For enterprises where system downtime incurs financial or safety implications—such as energy utilities or industrial automation—SOAP's overhead is often justified as a trade-off for enhanced consistency.

D. Security and Governance Considerations

Security remains a critical factor in protocol selection. SOAP integrates WS-Security, enabling message encryption, digital signatures, and end-to-end integrity verification. These capabilities are essential in multi-tenant and compliance-bound enterprises that require auditable communication channels. REST, while inherently dependent on transport-layer security (HTTPS), lacks standardized equivalents to SOAP's WS-* extensions. This limits its applicability in regulated sectors without custom security frameworks.

Nevertheless, the adoption of REST does not imply inadequate protection. Enterprises increasingly augment RESTful services with OAuth 2.0, JSON Web Tokens (JWT), and API gateways for authentication and authorization, achieving a security posture sufficient for most IoT and web-facing systems.

The choice of protocol therefore depends on balancing security rigor against operational agility—a decision often influenced by organizational risk tolerance and compliance obligations.

E. Strategic Implications for Enterprise IT

The findings suggest several actionable insights for IT decision-makers and enterprise architects:

Protocol Selection by Context: REST should be preferred for performance-sensitive, data-intensive, and scalable IoT operations. SOAP remains appropriate for inter-organizational communication requiring guaranteed delivery and schema validation.

Hybrid Deployment Models: Enterprises can combine REST for front-end and IoT interactions with SOAP for backend integration, ensuring both agility and compliance.

Transition Planning: For organizations migrating from monolithic SOAP systems to microservices, gradual integration of RESTful endpoints provides a manageable evolution path without service disruption.

Monitoring and Optimization: REST's stateless design enables more effective integration with observability tools (e.g., Prometheus, Grafana), facilitating proactive system tuning and cost control.

These strategies align with the architectural recommendations proposed by Guinard and Trifa [19], who advocate layered service integration and incremental modernization of enterprise IoT systems to achieve both scalability and governance.

F. Broader Impact

The comparison underscores an important insight for the pre-2018 enterprise landscape: performance optimization cannot be isolated from architectural context. While REST represents the dominant paradigm for cloud and IoT innovation, SOAP continues to define the backbone of many mission-critical enterprise integrations. The ability to balance these paradigms—through adaptive middleware and standardized gateways—will remain essential as organizations move toward service convergence in hybrid and multi-cloud environments.

7. Conclusion and Future Work

This study presented a detailed comparative evaluation of RESTful and SOAP-based Web services within the context of an enterprise IoT control system. Using a controlled indoor actuator network, both service models were benchmarked under identical operational conditions to measure latency, throughput, resource utilization, and payload efficiency. The results confirm that while both architectures maintain functional equivalence, they diverge significantly in performance characteristics and resource behavior—findings that have direct implications for enterprise system design, IoT deployments, and service modernization strategies.



A. Summary of Findings

The experimental results demonstrated that RESTful services outperform SOAP in all major performance metrics. REST achieved up to 40% lower latency, 42% higher throughput, and 25% lower CPU utilization under comparable workloads. These improvements are attributed to REST's statelessness, direct use of HTTP methods, and lightweight JSON serialization, which collectively reduce computational and transmission overhead.

SOAP, in contrast, exhibited increased latency and resource consumption due to XML message encapsulation and schema validation overhead. However, SOAP's structured message envelopes and reliance on WSDL contracts provided a degree of robustness and reliability that REST lacks natively. This distinction reinforces the premise that REST is more efficient for real-time actuator control, while SOAP is better suited for mission-critical transactional systems demanding guaranteed message delivery and schema conformance.

These results align closely with earlier theoretical and empirical studies conducted during the same era, including TinyREST [20] and Global Sensor Network middleware [21], which similarly highlighted REST's advantages in constrained IoT environments and SOAP's persistence in enterprise-grade reliability.

B. Enterprise Implications

From an enterprise perspective, the findings provide practical guidance for service architecture selection and system integration in mixed environments. Organizations seeking high scalability and responsiveness—such as smart infrastructure, logistics, or industrial IoT—can adopt RESTful APIs to minimize latency and operational cost. Meanwhile, enterprises that require strong security governance, contractual interoperability, and transactional precision can continue leveraging SOAP within their core systems.

Furthermore, the experiments reinforce the value of hybrid service architectures, in which REST interfaces handle time-sensitive actuator communication while SOAP manages configuration, data integrity, and auditing tasks. This dual strategy enables enterprises to optimize system performance without compromising compliance or reliability—a principle echoed in earlier SOA-to-IoT transition frameworks [18], [19].

For organizations migrating toward cloud-native ecosystems, REST's stateless design also aligns well with containerization and microservices orchestration, offering improved scalability under dynamic workloads. However, SOAP's structured contracts remain critical for enterprise-to-enterprise integration where formal service-level agreements (SLAs) and legacy compatibility are mandatory.

C. Broader Contributions

This work contributes to the ongoing discourse on web service optimization by providing empirical evidence rather than purely theoretical assertions. Unlike simulation-based evaluations prevalent before 2017, the study adopts a live testbed approach, measuring true system-level behavior in controlled yet realistic enterprise IoT conditions.

The inclusion of resource utilization metrics (CPU and memory) provides deeper insight into energy and cost implications—factors often neglected in early REST vs. SOAP comparisons. By situating the analysis within indoor actuator networks, the study extends Web service benchmarking beyond traditional web and mobile domains, offering new relevance for embedded and distributed control systems.

Additionally, the findings affirm the long-observed trade-off between performance and reliability: REST excels in communication agility, while SOAP remains indispensable for formalized data exchange. This dichotomy continues to define architectural decisions in hybrid enterprise systems, particularly those undergoing gradual modernization from legacy SOA infrastructures toward agile IoT-enabled environments.

D. Limitations

While the experiments were conducted under carefully controlled conditions, several limitations must be acknowledged. The study focused on single-server deployments within a LAN environment; results may differ under cloud-hosted or wide-area network (WAN) scenarios where factors such as latency variability and encryption overhead are more pronounced.

Moreover, the simulated actuator devices did not incorporate real sensor hardware or energy constraints, which may influence protocol efficiency in embedded implementations. Future studies should integrate physical IoT devices and wireless communication layers (e.g., Wi-Fi, Zigbee, or LoRaWAN) to capture real-world behavior.

E. Future Research Directions

Future work can extend this comparative framework in several promising directions:



Inclusion of Emerging Protocols: Investigate newer lightweight messaging paradigms such as MQTT, CoAP, and AMQP, which are designed specifically for IoT and could further reduce message overhead.

Hybrid Service Gateways: Develop adaptive middleware capable of dynamically switching between REST and SOAP based on workload characteristics, device constraints, or reliability requirements.

Security and QoS Evaluation: Incorporate advanced security mechanisms (e.g., OAuth2, WS-Security, TLS 1.3) and assess their effect on system performance.

Cloud-Native Benchmarking: Evaluate how both protocols perform within containerized and serverless architectures, extending the analysis to multi-tenant and distributed cloud environments.

Energy-Aware Service Design: Explore protocol efficiency in battery-powered IoT nodes, where energy consumption directly correlates with communication overhead.

Such studies would continue to bridge the gap between academic modeling and enterprise operational reality, contributing to the evolution of performance-optimized and secure IoT service frameworks.

F. Concluding Remarks

In conclusion, this research substantiates that RESTful services are more suitable for agile, scalable, and data-intensive IoT environments, whereas SOAP services maintain their relevance in reliable, security-sensitive enterprise applications. The decision between them should therefore be context-driven, informed by performance requirements, system constraints, and governance priorities.

By empirically validating these distinctions within an enterprise IoT context, this study not only complements earlier theoretical models [20], [21] but also provides a practical reference point for architects and engineers designing the next generation of interoperable, efficient, and secure web-based control systems.

References

- [1]. R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000.
- [2]. M. P. Papazoglou, *Web Services: Principles and Technology*, Pearson Education, 2008.
- [3]. N. Leavitt, "Are Web Services Finally Ready to Deliver?" *Computer*, vol. 37, no. 11, pp. 14–18, 2004.
- [4]. P. Pautasso, O. Zimmermann, and F. Leymann, "RESTful Web Services vs. 'Big' Web Services: Making the Right Architectural Decision," *Proc. 17th International World Wide Web Conference (WWW)*, 2008.
- [5]. L. Richardson and S. Ruby, *RESTful Web Services*, O'Reilly Media, 2007.
- [6]. E. Christensen, F. Curbera, G. Meredith, and S. Weerawarana, "Web Services Description Language (WSDL) 1.1," *W3C Note*, 2001.
- [7]. D. Booth, H. Haas, F. McCabe, E. Newcomer, M. Champion, C. Ferris, and D. Orchard, "Simple Object Access Protocol (SOAP) 1.2," *W3C Recommendation*, 2003.
- [8]. C. Pautasso, "RESTful Web Services: Principles, Patterns, and Emerging Technologies," *Software: Practice and Experience*, vol. 40, no. 12, pp. 1350–1375, 2010.
- [9]. T. Espada, P. Lacuesta, M. Lloret, and M. Canovas, "Performance Evaluation of SOAP and REST Web Services for Mobile Applications," *IEEE Latin America Transactions*, vol. 11, no. 2, pp. 291–297, 2013.
- [10]. J. Dede, K. Bratanov, and A. Fensel, "Performance Comparison of Web Service Architectures for Mobile Applications," *IEEE International Conference on Web Services (ICWS)*, 2014.
- [11]. J. Snell, D. Tidwell, and P. Kulchenko, *Programming Web Services with SOAP*, O'Reilly Media, 2002.
- [12]. A. P. Sheth, C. Bertram, and J. Miller, "Service-Oriented Architecture for the Internet of Things," *IEEE Internet Computing*, vol. 16, no. 2, pp. 52–61, 2012.
- [13]. A. Rahman, S. Khatun, and M. Rahman, "A Comparative Study of SOAP and REST Web Services in Android Environment," *International Journal of Computer Applications*, vol. 83, no. 18, pp. 35–42, 2013.
- [14]. M. Maleshkova, C. Pedrinaci, and J. Domingue, "Investigating Web Service Design Principles: A REST vs. SOAP Case Study," *IEEE International Conference on Web Services (ICWS)*, 2010.



- [15]. F. Belqasmi, R. Glitho, and N. El-Faddaghi, "RESTful Web Services for Service Provisioning in Next Generation Networks," *IEEE Communications Magazine*, vol. 49, no. 12, pp. 84–90, 2011.
- [16]. M. Toosi and R. Buyya, "Elastic Provisioning of Web Applications in Clouds: A Performance Evaluation," *Future Generation Computer Systems*, vol. 29, no. 5, pp. 1311–1321, 2014.
- [17]. R. Gupta and S. Chandra, "Experimental Analysis of SOAP and REST Web Services for Android-Based Mobile Applications," *International Journal of Computer Applications*, vol. 139, no. 11, pp. 1–6, 2016.
- [18]. P. Jamshidi, A. Ahmad, and C. Pahl, "Cloud Migration Research: A Systematic Review," *IEEE Transactions on Cloud Computing*, vol. 1, no. 2, pp. 142–157, 2013.
- [19]. D. Guinard and V. Trifa, *Building the Web of Things: With Examples in Node.js and Raspberry Pi*, Manning Publications, 2016.
- [20]. T. Luckenbach, P. Gober, S. Arbanowski, A. Kotsopoulos, and K. Kim, "TinyREST: A Protocol for Integrating Sensor Networks into the Internet," *Proc. REALWSN Workshop*, 2005.
- [21]. K. Aberer, M. Hauswirth, and A. Salehi, "The Global Sensor Networks Middleware for Efficient and Flexible IoT Integration," *IEEE Communications Magazine*, vol. 49, no. 3, pp. 50–57, 2012.

